Senior Engineer Deep Dive and Interview Guide

Java, API management, distributed systems, Kubernetes, security, observability and AI-assisted engineering

Broadcom_Senior_Engineer_Deep_Dive_and_Interview_Guide

A study reference tied to a working project: every topic links to code, tests or clearly labelled conceptual material in the accompanying repository. Part 1 is the technical reference; Part 2 contains 120+ interview questions with model answers (40+ scenario-based), trade-offs, failure modes and likely follow-ups.

How to use it: read the topic chapters, then answer each question aloud before reading the model answer. Where an answer cites project evidence (a test name, file or measured number) you can open it in the repository and demonstrate it.

Honesty note: measured numbers come from a single shared-CPU laptop and are relative only; items not executed in the build environment are labelled as such.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Contents

Part 1 - Technical reference **3**
1. Java and Spring Boot **3**
1.1 JVM, memory and garbage collection 3
1.2 Concurrency and thread pools 3
1.3 Spring dependency injection and configuration 3
1.4 Transaction management and Hibernate behaviour 4
1.5 REST API design, errors, security, testing 4
2. API management **5**
2.1 Gateway architecture and control vs data plane 5
2.2 Routing, filtering, authentication, authorization 5
2.3 Rate limiting and policy propagation 5
2.4 Versioning, backward compatibility and lifecycle 5
2.5 Observability and analytics for APIs 5
3. Distributed systems **6**
4. Kubernetes and cloud **7**
4.1 Docker image design 7
4.2 Kubernetes resources, probes, limits, autoscaling 7
5. Observability and performance **8**
6. Security **9**
6.1 OAuth 2.0, OIDC and JWT 9
6.2 OWASP, SSRF, API authorization 9
6.3 TLS, rate limiting, scanning, containers 9
7. AI-assisted engineering **10**
7.1 Specification-driven development and BMAD-style workflows 10
7.2 Claude Code and similar assistants: repository instructions and skills 10
7.3 Context engineering and token optimization 10
7.4 Multi-agent orchestration 10
7.5 AI-generated code review, safe refactoring, upgrades and patching 10
Part 2 - Interview questions with model answers **11**
Java and the JVM 12
API management 14
Rate limiting and resilience 16
Distributed systems 17
Security 19
Kubernetes 20
Cloud architecture 22
Observability 23
Performance engineering 25
Testing 26
AI-assisted development 28
Context engineering 29
Multi-agent workflows 30
Troubleshooting scenarios 32
Technical leadership 33
Appendix - quick reference **35**

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Part 1 - Technical reference

Each chapter explains the concept, then ties it to code, tests or clearly labelled conceptual material in the accompanying repository, so you can **show** rather than just describe. Paths are relative to the repository root.

1. Java and Spring Boot

1.1 JVM, memory and garbage collection

- **Heap vs non-heap:** heap holds objects (young + old generations in G1 regions); non-heap holds metaspace (class metadata), thread stacks, code cache, direct buffers. A container memory limit must cover **both**: with `-XX:MaxRAMPercentage=75` a 512 MiB limit gives ~384 MiB heap and ~128 MiB for everything else - an `OOMKilled` pod with a healthy heap usually means the non-heap share was too small (`docs/troubleshooting/09-pods-restarting.md`).
- **G1 (default):** region-based, mostly-concurrent marking, mixed collections; pause goals not guarantees. **Humongous allocations** (objects >= half a region, e.g. large byte arrays) are allocated directly in old space and cause frequent mixed GCs - the reason the gateway caps request bodies and pre-sizes buffers (exercise 06).
- **What to watch:** heap **after** GC (the floor of the sawtooth) trending up = leak or growing working set; allocation rate; pause time share (`rate(jvm_gc_pause_seconds_sum[1m])`); promotion rate.
- **Tools:** `jcmd GC.heap_info / GC.class_histogram / GC.heap_dump`, JFR (`jcmd JFR.start`), `jstat -gcutil`, `-Xlog:gc*`, Eclipse MAT dominator tree.
- **In the project:** `ExitOnOutOfMemoryError` (restart beats a half-dead JVM); every cache/map is bounded (limiter eviction at 200k keys, telemetry buffers 2000, response cap 10 MiB).

1.2 Concurrency and thread pools

- **Platform threads** are expensive (~1 MiB stack, OS scheduled); pools size concurrency. **Virtual threads (Java 21)** are cheap and mounted on a small carrier pool; blocking I/O unmounts them, so thread-per-request scales to many thousands of concurrent calls. The pool you tune moves to the **downstream resource**: DB connections, Redis, semaphores.
- **Pinning:** a virtual thread blocked inside `synchronized` (or native frames) pins its carrier. Avoid `synchronized` around I/O; use `ReentrantLock`/`Semaphore`. Diagnose with `-Djdk.tracePinnedThreads=full`.
- **Backpressure:** unbounded queues hide overload. The gateway uses a per-route `Semaphore` with immediate rejection (503 + Retry-After) instead of queueing.
- **Safe sharing:** immutable records + `AtomicReference` swap (route table), per-key monitors (limiter), `LongAdder` counters (metrics). `InMemoryRateLimiterTest` proves exactness under 64 threads.
- **Visibility and atomicity:** `volatile` gives visibility not atomicity; check-then-act needs a lock, CAS or a database constraint (policy version numbers use a unique key).

1.3 Spring dependency injection and configuration

- Constructor injection (final fields, testable without Spring); `@ConfigurationProperties` records with defaults for typed config (`GatewayProperties`, `SecurityProperties`); `@ConditionalOnProperty` to keep dev-only beans (dev token issuer) off by default.
- **Proxy-based AOP gotchas:** `@Transactional`, `@PreAuthorize`, `@Cacheable` apply only to calls that go **through the proxy** - a self-invocation bypasses them. Real example: `SnapshotService.current()` calls

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

`build()` internally, so `current()` itself carries `@Transactional(readOnly=true)` (the first run failed with `LazyInitializationException`).

- **Profiles and environment:** the `local` profile (H2, dev IdP, seed data) is demo-only; production reads secrets from the environment/secret manager.

1.4 Transaction management and Hibernate behaviour

- **Boundaries** on services; reads `readOnly=true`; `open-in-view=false` so lazy loading cannot leak to the web layer. Rollback is for `RuntimeException`/`Error` by default - audit rows for rejected operations need `noRollbackFor=ApiException` (`ApiService.addTarget`).
- **Propagation:** `AuditService` uses `MANDATORY`: it cannot run outside the transaction of the change it records, making change + audit atomic.
- **Hibernate:** first-level cache per session; dirty checking flushes changes at commit; `IDENTITY` ids disable insert batching; `@Version` gives optimistic locking (`409` on conflict); `join fetch` solves N+1 but fetching two bags at once throws `MultipleBagFetchException` - fetch one collection per query.
- **Isolation:** MySQL InnoDB default REPEATABLE READ with MVCC; use unique constraints as the final arbiter of races (`(api_id, version_no)`).

1.5 REST API design, errors, security, testing

- **Design:** resource nouns, correct status codes (400 malformed, 422 semantic, 409 conflict), RFC 7807 problem bodies with stable `code` + `correlationId`, pagination caps, whitelisted sorts, idempotency keys for retried creates, optimistic locking for updates.
- **Security:** resource-server JWT validation, `@PreAuthorize` + object-level `AccessGuard`, stateless sessions, CSRF disabled **because** no cookies are used, CORS allowlist.
- **Testing pyramid:** unit (validators, algorithms) -> `@SpringBootTest` + MockMvc (real filter chain, H2) -> gateway tests on a random port with a real HTTP backend -> Testcontainers for MySQL/Redis (need Docker) -> Playwright e2e. Fake clocks instead of sleeps.
- **Performance troubleshooting loop:** reproduce -> measure (percentiles, not averages) -> profile -> change one thing -> re-measure -> record method and caveats.

Java/Spring pitfall	Symptom	Project example / guard
Self-invoked @Transactional	LazyInitializationException, no tx	SnapshotService.current annotated itself
@Async/@Cacheable on private/self-called	Annotation silently ignored	Prefer explicit service boundaries
Unbounded cache/map	Heap climbs until OOMKilled	Limiter key eviction + bounded buffers
Catch-all exception returning default	Security failure swallowed	Fail closed, ApiException + handler
Check-then-act without constraint	Duplicate versions	Unique key + 409 + concurrency IT

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

2. API management

2.1 Gateway architecture and control vs data plane

- **Data plane:** the request path - routing, authN/Z, limits, proxying, resilience, telemetry; must be fast, stateless, horizontally scalable, and survive control-plane loss. **Control plane:** configuration, workflows, identities, audit, analytics; transactional, lower QPS, can be briefly unavailable.
- **Why split:** different scaling, failure domains, release cadence and security blast radius. Cost: configuration propagation delay and a versioned contract (`docs/api-contracts.md`).
- **Gateway patterns:** edge gateway (north-south), internal gateway/mesh (east-west), BFF. This project is an edge/API gateway; a service mesh solves mTLS and retries between services and complements, not replaces, API management.

2.2 Routing, filtering, authentication, authorization

- **Routing:** longest-prefix match on base path via an immutable hash map walked segment by segment (O(segments)); path rewriting strips the base path; weighted targets with health and breaker awareness.
- **Filter chain order matters:** cheap anonymous-safe checks first (route, method, IP, path), then authentication, authorization, rate limit, size, bulkhead, proxy. Unauthenticated callers learn only generic errors.
- **Authentication choices:** API keys (simple, per-consumer, revocable, must be hashed and expiring), JWT/OAuth2 client credentials (standard, short-lived, verifiable offline), mTLS (strong service identity). Authorization: RBAC + scopes + object-level checks.

2.3 Rate limiting and policy propagation

- Algorithms, local vs Redis, headers, failure modes: `docs/rate-limiting.md` (fixed window boundary burst, sliding-window counter, token bucket, atomic Lua, fail LOCAL vs CLOSED).
- **Propagation:** versioned snapshots, ETag polling, validate-then-swap, last-known-good on disk, bounded not instantaneous (`docs/policy-propagation.md`). Alternatives: push, broker fan-out, direct DB reads - each with different failure/consistency trade-offs (ADR-0003).

2.4 Versioning, backward compatibility and lifecycle

- **Versioning:** URI major version for the management API; per-version base paths and lifecycle for managed APIs. Additive-only changes inside a major; a new required field is a breaking change even though it "adds" something.
- **Lifecycle:** DRAFT -> ACTIVE -> DEPRECATED -> INACTIVE with a state machine (`Lifecycle.canMoveTo`); deprecation with `Deprecation`/`Sunset` headers and consumer notification; deactivation only after traffic drains (use traffic metrics per version).
- **Compatibility techniques:** tolerant readers (`ignoreUnknown`), expand/contract migrations, consumer-driven contract tests.

2.5 Observability and analytics for APIs

- RED metrics (rate, errors, duration) per route; percentiles via histograms; status-class distribution; protection signals (429/401/504, retries); config-change correlation; deterministic incident rules that show evidence.
- Cardinality discipline: labels only from the registry (bounded); per-consumer analytics belong in logs/traces or a dedicated store.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

3. Distributed systems

Concept	Principle	Where in the project
Timeouts	Every call bounded; use an **end-to-end budget**, not per-attempt, so retries cannot multiply latency	ProxyService.forward (policy.timeoutMs)
Retries	Only idempotent operations; exponential backoff with **jitter**; bounded attempts; retry budgets avoid storms	safe methods only, <=3, full jitter
Circuit breaker	Stop calling a failing dependency; half-open probes; consecutive-failure vs failure-rate designs	CircuitBreaker (consecutive), ADR-0006
Bulkhead	Isolate resources per dependency/route so one cannot exhaust all	per-route Semaphore
Backpressure	Reject early rather than queue unboundedly; propagate pressure to callers	503 route_saturated + Retry-After
Idempotency	Same request twice = same effect; keys + request hash + stored response	IdempotencyService, telemetry batches
Eventual consistency	Define the window and what clients may observe in it	Policy propagation (poll + cache)
Delivery guarantees	At-most-once (may lose), at-least-once (may duplicate - needs idempotent receiver), exactly-once = at-least-once + idempotency/transactions	Telemetry: at-least-once + dedupe
Failure isolation	Fail static vs fail closed; degrade rather than collapse	Last-known-good config; fail-closed publish
Scalability	Stateless data plane, shard hot state (Redis keys with hash tags), avoid global locks	Gateway replicas + Redis limiter

> **Note:** Message delivery guarantees: Kafka offers at-least-once by default and exactly-once only within Kafka transactions plus idempotent producers/consumers; RabbitMQ offers at-least-once with acks and DLQs. In both cases consumers must be idempotent. This project deliberately uses no broker (ADR-0007) and explains the triggers that would justify one.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

4. Kubernetes and cloud

4.1 Docker image design

- Multi-stage builds (build tools never ship), layered Spring Boot jars (dependency layer cached), minimal JRE base, **non-root numeric uid**, read-only root filesystem, no secrets in layers, healthcheck, `MaxRAMPercentage`, `ExitOnOutOfMemoryError`; scan (Trivy), lint (hadolint), pin by digest, sign (cosign).

4.2 Kubernetes resources, probes, limits, autoscaling

Resource / setting	Purpose	Project value
Deployment RollingUpdate	Zero-downtime rollouts	maxUnavailable 0, maxSurge 1
startup / readiness / liveness	Slow JVM start; traffic gating; restart on deadlock	actuator liveness+readiness; gateway readiness needs snapshot; Redis excluded
requests / limits	Scheduling and noisy-neighbour control; JVM sizing	gateway 500m/384Mi -> 2/512Mi
HPA	Scale on CPU (or custom metrics such as RPS)	gateway 3-20 @60% CPU, control plane 2-6 @70%
PDB	Voluntary disruption safety	gateway maxUnavailable 1; CP minAvailable 1
preStop + terminationGrace	Drain: endpoints removed before SIGTERM	10 s sleep + 25 s drain < 40 s
NetworkPolicy	Default deny; least-privilege flows	backends reachable only from the gateway
Pod Security (restricted)	Block privileged pods	namespace label + securityContext
ServiceAccount	Workload identity; no API token if unused	automountServiceAccountToken false

- **Networking:** Service (ClusterIP) load balances across ready pods; Ingress terminates TLS and routes by host/path; DNS-based discovery; NetworkPolicies need a CNI that enforces them.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- **Helm:** templated manifests + values; `--atomic` upgrades, `helm rollback`, `checksum/config` to roll pods on config change; secrets by reference.
- **CI/CD:** build once, scan, sign, promote the same artifact; deploy via GitOps or an OIDC-federated pipeline.
- **Secrets management:** external secret manager synced by External Secrets/CSI; avoid env-var exposure where `exec` is possible; rotate; never commit.
- **Cloud choices:** AWS primary (EKS, Aurora/RDS MySQL, ElastiCache, ALB+ACM, Secrets Manager, IRSA, ECR) with Azure/GCP equivalents; local substitutes are not equivalent (failover, quotas, networking differ) - `docs/kubernetes-and-helm.md`.

> **Caution:** Honesty: none of the Docker/Kubernetes/Helm assets in the repository were built or applied in the build environment (no Docker/kubectl/helm). Treat them as reviewed designs until validated in a real cluster.

5. Observability and performance

- **Logs, metrics, traces:** logs = discrete events with context (use structured JSON + correlation ids); metrics = aggregated numbers (cheap, alertable, low detail); traces = per-request causality across services (sampled). Use metrics to detect, traces/logs to explain.
- **OpenTelemetry:** vendor-neutral API/SDK and wire protocol (OTLP); Micrometer Tracing bridges Spring to OTel; the Collector can batch, redact, tail-sample and route to multiple backends.
- **Prometheus/Grafana:** pull model with scrape targets, histograms for latency (`histogram_quantile` over `_bucket`), recording/alert rules, dashboards as code (provisioned JSON in `observability/grafana`).
- **Percentiles:** averages hide tail latency; aggregate histograms, not percentiles (percentiles of percentiles are meaningless); choose buckets near your SLO; p99 needs enough samples.
- **Cardinality:** every unique label combination is a series; unbounded labels (user id, path, IP) explode memory. Budget series = routes x classes x buckets; push detail to logs/traces.
- **Sampling:** head (cheap, random, can miss rare errors) vs tail (keeps errors/slow traces, needs a Collector holding spans briefly).
- **JVM and DB bottlenecks:** GC pause share, heap after GC, thread dumps and pinning, Hikari pending/usage, slow query log + `EXPLAIN ANALYZE`, N+1, missing indexes.
- **Load testing:** open-model arrival rate avoids coordinated omission; warm up; compare to a baseline; separate generator from SUT; report percentiles, errors and resource use with caveats (`docs/performance.md`).
- **SLOs and incident response:** SLI (good events / valid events), SLO target, error budget, burn-rate alerts (fast + slow windows); incident roles, mitigation before diagnosis, blameless review (`docs/runbooks`, `docs/leadership`).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

6. Security

6.1 OAuth 2.0, OIDC and JWT

- **OAuth 2.0** delegates authorization (access tokens); **OIDC** adds authentication (ID token, discovery, JWKS, userinfo). Flows: authorization code + **PKCE** for SPAs/native, client credentials for service-to-service; avoid implicit and password grants.
- **JWT validation checklist:** verify signature with the expected algorithm/key (never accept `alg:none`, never let the token choose the key type), `iss`, `aud`, `exp`, `nbf`, optional `typ`; allow small clock-skew leeway; rotate keys via JWKS (`kid`); keep lifetimes short; revocation needs short TTL or introspection.
- **In the project:** Spring resource server validates signature/issuer/expiry (HS256 in dev, `issuer-uri` in production); gateway tests cover expired, wrong issuer, forged signature, future `nbf`, `alg:none`, missing role.

6.2 OWASP, SSRF, API authorization

- **OWASP API Top 10:** BOLA is the most common - always check object ownership server-side; broken function-level authorization; unrestricted resource consumption (limits!); SSRF; security misconfiguration; improper inventory (registry + lifecycle).
- **SSRF defence in depth:** allowlist hosts (not blocklist), block link-local/metadata/any-local ranges, forbid credentials and redirects, resolve and verify addresses at connect time (DNS rebinding), restrict egress with NetworkPolicy/egress proxy (ADR-0008).
- **Secrets:** never in git/images/logs; hash high-entropy tokens (SHA-256), slow-hash passwords (argon2/bcrypt); show once; rotate; constant-time compare.

6.3 TLS, rate limiting, scanning, containers

- **TLS:** terminate at ingress, TLS 1.2+/1.3, HSTS, certificate automation, mTLS inside the cluster for service identity.
- **Rate limiting as abuse protection:** per consumer + per IP for anonymous traffic; 429 with Retry-After; fail-closed vs fail-open decision for the limiter backend.
- **Dependency scanning:** `npm audit`, Trivy/OSV, SBOM, triage by reachability and exposure; **accept with owner and expiry** when no fix exists (the project's real `braces` dev-tooling finding).
- **Container security:** non-root, read-only FS, drop capabilities, seccomp, PSA restricted, signed minimal images, no `latest` tags.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

7. AI-assisted engineering

7.1 Specification-driven development and BMAD-style workflows

- **Idea:** write the intent (objective, acceptance criteria, security, failure behaviour, rollout) first, derive design and tasks from it, implement in small verified steps. The spec is the contract between humans and assistants.
- **BMAD-style:** role-based phases (analyst, product manager, architect, scrum master, developer, QA) producing artifacts that feed the next phase; **spec-kit style:** specify -> plan -> tasks -> implement. Methodologies evolve - keep the artifacts (`specs/`) tool-independent.
- **Human gates:** approve the spec and design, review security-sensitive code, decide releases; assistants draft, humans decide.

7.2 Claude Code and similar assistants: repository instructions and skills

- **Repository instructions** (`CLAUDE.md`, `AGENTS.md`, Copilot instructions) hold stable, short rules and pointers; **skills/modules** hold task-specific procedures loaded on demand (`ai-engineering/*.md`). Keep instructions free of secrets and of facts derivable from code.
- **Verification loop:** assistants must run tests, read outputs and report skipped/blocked checks honestly; never claim unrun success.
- **Permissions:** least privilege, deny destructive operations by default, human approval for pushes and production actions.

7.3 Context engineering and token optimization

- Always-loaded concise context + on-demand modules; repository map; read the smallest relevant range; summarise completed work into handoff notes; ADRs preserve decisions; split large tasks; detect missing context (invented APIs, no file citations).
- **Measure, do not guess:** same tasks, same model, fresh sessions; compare input/output tokens, tool calls and pass rate from the tool's usage report; cheaper but less correct is a regression.

7.4 Multi-agent orchestration

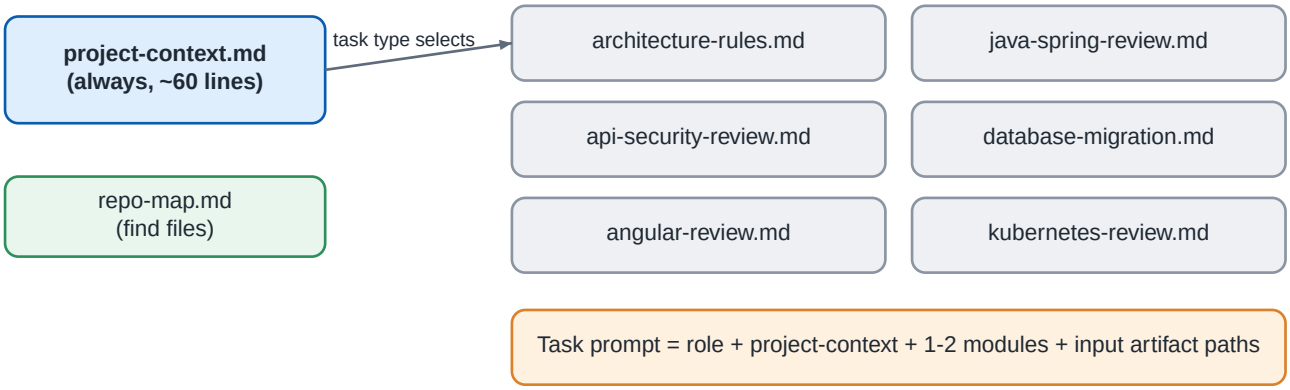
- Specialised roles with file ownership, frozen shared contracts, dependency-aware sequencing, independent review gates, evidence-bearing handoffs, human approval for sensitive changes; conflicts escalate to the architecture role, then a human.
- **Failure modes:** conflicting architectural assumptions, silent contract drift, self-approval, context bloat from transcripts, duplicated edits on shared files - each addressed by ownership, contracts and gates (`ai-engineering/multi-agent/`).

7.5 AI-generated code review, safe refactoring, upgrades and patching

- **Review stance:** untrusted until read, run and attacked; check invented APIs, missing invariants, mirror tests, concurrency/security blind spots (`review-examples/examples.md`).
- **Safe refactoring:** characterise with tests first, small steps, compare performance, keep diffs reviewable, rollback by revert (and be honest when a refactor happened before the tests existed).
- **Dependency upgrades and vulnerability patching:** triage with evidence (reachability/exposure/fix availability), add regression tests first, smallest bump, verify, plan rollback; distinguish confirmed findings from illustrative ones; never invent CVE ids or scores (`docs/governance/vulnerability-remediation.md` records a real `npm audit` finding and a real `java-jwt` 4.4.0 -> 4.6.2 upgrade).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- **Human approval and validation:** every release decision and every security exception has a named human owner.



Hand off summaries + paths (not transcripts); measure tokens from the tool usage report before claiming savings.

Figure 1. Modular context loading: one short always-loaded file plus task-selected modules.

Part 2 - Interview questions with model answers

This part contains **122 questions** grouped by topic; **53 are scenario-based** (marked [Scenario]). Each model answer gives the technical reasoning, trade-offs and failure modes; "Likely follow-ups" lists what a good interviewer asks next; "In this project" ties the answer to code, tests or documents you can demonstrate.

Topic	Questions	Scenario-based
Java and the JVM	8	4
API management	8	4
Rate limiting and resilience	8	3
Distributed systems	8	2
Security	8	2
Kubernetes	8	5
Cloud architecture	8	3
Observability	8	3
Performance engineering	8	3

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Topic	Questions	Scenario-based
Testing	8	1
AI-assisted development	8	4
Context engineering	7	2
Multi-agent workflows	7	3
Troubleshooting scenarios	10	10
Technical leadership	10	4
Total	**122**	**53**

> **Note:** How to practise: cover the answer, speak yours for 2-3 minutes (reasoning, trade-off, failure mode, example), then compare. Prepare one concrete story per topic from this project - and be honest about what was not verified (see the validation report).

Java and the JVM

Q1 [Scenario]. A gateway pod is OOMKilled although the heap dashboard shows 60% usage. What is going on and how do you fix it?

The container limit covers more than heap: metaspace, thread stacks, code cache, direct buffers (Netty/NIO), GC structures and the native allocator. With MaxRAMPercentage=75 a 512 MiB limit leaves ~128 MiB for everything else, and spikes (TLS buffers, many threads, large buffered bodies) can exceed it while the heap looks fine. Evidence: `kubectl describe pod` (OOMKilled, exit 137), `jcmd VM.native_memory summary` with NMT enabled, non-heap and buffer metrics. Fix: raise the limit or lower MaxRAMPercentage (e.g. 60-70%), cap buffers, avoid per-request large arrays; do not just raise the liveness threshold. Prevent with load tests that assert RSS stays below ~85% of the limit.

Likely follow-ups: How do you enable NMT? What is the difference between RSS and committed heap? Why not set -Xmx?

In this project: Exercise 09; gateway limits 512 Mi with MaxRAMPercentage=75; body/response caps bound buffering.

Q2 [Scenario]. How would you troubleshoot high JVM memory consumption in production?

Start with symptoms: is it heap, non-heap or container RSS, and is it growing (leak) or steady (working set)? Look at heap **after** GC over hours, GC frequency, class histogram twice 10 minutes apart (`jcmd GC.class_histogram`), then a heap dump analysed in MAT (dominator tree, leak suspects) - taken from one replica taken out of rotation if dumps pause the JVM. Typical causes: unbounded maps/caches keyed by user input (per-IP limiter state), listeners never removed, big response buffers, thread-local leaks. Fix the unbounded structure (bound + eviction + test), then confirm with a soak test. Raise memory last.

Likely follow-ups: What if the dump is too large? How do you dump safely? What about off-heap leaks?

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

In this project: Exercise 05; limiter keys evicted above 200k; telemetry buffers capped at 2000.

Q3 [Scenario]. Latency spikes every few seconds and CPU is high. How do you decide whether GC is the cause?

Correlate spike timestamps with GC pause events (`-Xlog:gc*`, `jvm_gc_pause_seconds_max`) and compute pause share `rate(jvm_gc_pause_seconds_sum[1m])`. If pauses line up, inspect causes (Humongous Allocation, Allocation Failure, Evacuation Failure) and allocation rate in JFR. If they do not line up, profile CPU (async-profiler flame graph) for hot code, check CPU throttling in the cgroup, and thread dumps for lock contention. G1 fixes: reduce humongous allocations (cap/pre-size buffers), increase heap to 2-3x live set, avoid explicit System.gc(). Only then tune flags.

Likely follow-ups: Why can CPU throttling look like GC trouble? When would you switch to ZGC?

In this project: Exercise 06 (9 MiB bodies producing humongous allocations); limits on request size in policy validator.

Q4. Explain virtual threads and when they hurt.

Virtual threads are JVM-scheduled threads mounted on a small pool of carrier threads; when they block on I/O they unmount, so thread-per-request code scales to many thousands of concurrent requests without reactive code. They hurt when (1) work is CPU-bound (no benefit), (2) a thread pins its carrier by blocking inside `synchronized` or native frames, (3) code uses unbounded per-thread caches or ThreadLocals heavily, and (4) the real bottleneck is a finite downstream resource (DB pool, Redis, semaphores) - virtual threads simply make the queue invisible and longer unless you add limits.

Likely follow-ups: How do you detect pinning? Should you pool virtual threads?

In this project: ADR-0002; gateway uses spring.threads.virtual.enabled with a per-route semaphore bulkhead.

Q5. What are the common @Transactional pitfalls in Spring?

Proxy-based: self-invocation, private/final methods and non-Spring-managed calls bypass the transaction; only RuntimeException/Error roll back by default (checked exceptions commit unless configured); readOnly is a hint; propagation REQUIRES_NEW can deadlock a pool if the outer transaction holds the only connection; long transactions hold connections and locks; LazyInitializationException appears when lazy associations are touched after the session closes. Fixes: keep service boundaries explicit, annotate the public entry point, configure rollback rules, map to DTOs inside the transaction, set open-in-view=false.

Likely follow-ups: How does readOnly affect Hibernate? What is the difference between MANDATORY and REQUIRED?

In this project: SnapshotService.current needed its own @Transactional; AuditService uses MANDATORY; ApiService.addTarget uses noRollbackFor=ApiException.

Q6. How do you prevent and detect N+1 queries with Hibernate?

Prevent by designing read models: fetch what the view needs with a join fetch/entity graph or DTO projection and page results; avoid lazy collections in loops. Detect with SQL logging in tests, `hibernate.generate_statistics`, assertion libraries counting statements, and slow-query/EXPLAIN review. Be careful: fetching two collections (bags) in one query throws MultipleBagFetchException and multiplies rows; use separate queries or batch fetching (`default_batch_fetch_size`).

Likely follow-ups: Join fetch with pagination? What does open-in-view do?

In this project: ApiDefinitionRepository.findAllWithVersionsByLifecycle; open-in-view disabled.

Q7. With virtual threads, how do you size thread pools and the database pool?

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

You stop sizing request thread pools; concurrency is limited by downstream resources. Size the DB pool to what MySQL can execute efficiently (often ~ (cores x 2) + effective spindles for the DB server, validated by load test), set a short connection timeout so overload fails fast, and use semaphores/bulkheads at the edge to cap in-flight work. Monitor `hikaricp_connections_pending` and acquire time; if threads wait on the pool, adding more virtual threads makes it worse.

Likely follow-ups: What happens at 10x load? How do you pick connection-timeout?

In this project: Hikari max 15, connection-timeout 3 s; per-route semaphore in the gateway.

Q8 [Scenario]. Design an idempotent "create API" endpoint that clients may retry after a timeout.

Require an Idempotency-Key header. In one transaction: look up the key; if present with the same request hash return the stored response (with a replay marker); if present with a different hash return 422; otherwise perform the business action and store key + hash + response. Store the record in the same transaction as the change so it exists iff the change committed. Add a unique key on the idempotency table to resolve concurrent duplicates (loser retries or gets the stored result) and a TTL cleanup. Not every operation needs it - natural keys (unique slug) already give 409 on duplicates.

Likely follow-ups: What about concurrent identical requests? How long do you keep keys?

In this project: IdempotencyService; test idempotencyKeyReplaysSameResponseAndRejectsDifferentBody.

API management

Q9 [Scenario]. How would you design an API gateway for thousands of backend services?

Separate control and data planes. The control plane stores registry/policy state and publishes versioned configuration; gateways stay stateless and keep an immutable in-memory route table (hash lookup per path segment) so lookup cost is independent of route count. Shard configuration (per team/region/gateway group) so snapshots stay small, use delta or sharded snapshots beyond a few MB, and push a version hint so polling is cheap (ETag/304). Isolate failure domains (bulkheads per route, breakers per target), bound all per-route state, and keep metric labels bounded (route ids, never paths). Operate with canary configuration rollout, last-known-good, and per-route SLOs. Consider multiple gateway fleets (partner vs internal) instead of one giant one.

Likely follow-ups: How do you handle tenants with very different traffic? What limits snapshot size? How do you roll out config safely?

In this project: RouteTable test (20k routes), ADR-0001/0003, bounded labels.

Q10. Explain control plane vs data plane and why the split matters.

The data plane handles live requests: it must be fast, stateless, horizontally scalable and independent of anything slow. The control plane manages configuration, identities and workflows: transactional, lower QPS, can be briefly unavailable. The split gives independent scaling and release cadence, limits blast radius (a bad admin query cannot slow traffic), and enables fail-static behaviour. Costs: eventual consistency of configuration, a versioned contract, and two things to operate. Anything that must be strongly consistent on the hot path (revocation latency, quotas) needs special design.

Likely follow-ups: What would you do if revocation must be instant? What data crosses the boundary?

In this project: ADR-0001; snapshot + telemetry contracts in docs/api-contracts.md.

Q11. How do you make route matching fast and correct?

Precompute an immutable hash map keyed by base path at config-apply time and match by walking the request path from longest to shortest prefix at segment boundaries (one hash lookup per segment). Correctness: match on segment boundaries (`/orders/v1` must not match `/orders/v10`), normalise/reject dot-segments and encoded traversal before matching, make precedence explicit (longest prefix), reject

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

overlapping registrations at creation (unique base path), and test with large route sets.

Likely follow-ups: What about regex or host-based routing? How do you test precedence?

In this project: RouteTable, PathSafetyTest, unique base_path constraint.

Q12 [Scenario]. You must deprecate an API version used by 40 partners. What is your process?

Mark the version DEPRECATED in the registry (still routable), publish a sunset date, add Deprecation/Sunset headers at the gateway, notify subscription contacts, and track per-version traffic by consumer to find laggards. Offer a migration guide and compatibility shims where cheap. Escalate to individual partners with the usage data as the sunset nears, then deactivate (INACTIVE) only when traffic is negligible or the date passes by agreement; keep the ability to reactivate quickly. Audit every transition.

Likely follow-ups: What if a key partner refuses? How do you measure usage per version?

In this project: Lifecycle state machine, ADR-0010, per-route metrics.

Q13 [Scenario]. How would you design API authentication for multiple consumer applications?

Give each consumer an identity (client id) and credentials appropriate to risk: OAuth2 client-credentials JWTs verified offline against the IdP JWKS for strong, short-lived tokens; API keys (random 256-bit, hashed at rest, prefix lookup, expiry, rotation, revocation) for simple partners; mTLS for high-assurance integrations. Bind credentials to subscriptions (consumer x API x plan), authorize by scopes/roles plus object-level rules, enforce per-consumer limits, never forward credentials to backends (assert identity via gateway headers), audit issuance/revocation, and provide a rotation path with overlapping validity.

Likely follow-ups: How do you rotate without downtime? JWT vs API key trade-offs? How do you revoke a JWT?

In this project: ConsumerService, Authenticator, ADR-0005.

Q14 [Scenario]. How would you safely propagate policy changes across gateway instances?

Version policies, validate them before activation, require review for risky changes, publish immutable snapshots, and let gateways pull with conditional requests. Each gateway validates, builds, swaps atomically and keeps last-known-good; invalid snapshots never replace valid ones. Make the consistency window explicit (poll interval + cache staleness), monitor snapshot age and applied version per gateway, support instant rollback by creating a new version from an old one, and consider canary groups for risky changes. State clearly that propagation is not instantaneous; for emergencies use disable/revoke plus edge rules.

Likely follow-ups: How do you canary a policy? What if gateways disagree during rollout? How do you prove it propagated?

In this project: Policy workflow, docs/policy-propagation.md, measured ~7-8 s locally.

Q15. Why separate submit, validate, approve and apply for policy changes?

Validation catches structural/safety errors early and cheaply (and is available as a dry run); approval by a different person catches intent errors validation cannot (limit too low, wrong auth); versioning gives history and rollback; apply is automatic and idempotent; auditing records who/what/when in the same transaction. The workflow adds minutes of latency to a change, so keep an audited emergency path (revert) for incidents rather than weakening the normal path.

Likely follow-ups: Who should approve? What about auto-approving low-risk changes?

In this project: PolicyService, self_approval test, revert.

Q16. Why does the gateway filter order matter?

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Cheap checks first, expensive/stateful later, and unauthenticated callers should learn as little as possible: route -> method -> IP -> path safety -> authN/Z -> rate limit -> size -> bulkhead -> proxy. Rate limiting after authentication lets limits be per consumer; size checks after authentication avoid reading bodies from anonymous clients; the bulkhead sits right before the expensive proxy call. Wrong order creates DoS amplifiers (e.g. reading a 10 MiB body before checking auth) or information leaks (different errors for unknown vs unauthorized resources).

Likely follow-ups: Where would WAF/IP reputation sit? Where do you put request validation against schema?

In this project: GatewayFilter documented order and tests.

Rate limiting and resilience

Q17 [Scenario]. How would you enforce distributed rate limits?

Use a shared atomic counter store (Redis) with one Lua script per decision so read-modify-write cannot interleave across gateways, use the store's clock to avoid skew, give keys TTLs, and hash-tag keys for cluster slot affinity. Choose the algorithm per need (token bucket for steady rate plus burst). Add a client timeout of tens of milliseconds, a short back-off after failures, and an explicit failure policy: fall back to per-instance limits scaled by expected replicas (availability first) or reject (strictness first). Expose X-RateLimit headers, 429 + Retry-After, metrics and events for degraded mode.

Likely follow-ups: What about hot keys? Multi-region? How do you test it?

In this project: RedisRateLimiter, ResilientRateLimiter, docs/rate-limiting.md (Redis IT needs Docker).

Q18. Compare fixed window, sliding window and token bucket.

Fixed window counts per aligned interval: cheapest but allows up to 2x the limit around a boundary. Sliding-window counter blends the previous and current windows: removes most of the boundary burst with O(1) memory but is approximate. Token bucket refills at a steady rate with a capacity: smooth average rate plus controlled burst, best default for APIs. Leaky bucket shapes (queues) rather than meters - rarely right for a gateway because queuing consumes resources. Sliding log is exact but O(n) memory.

Likely follow-ups: Which would you use for login endpoints? How do you pick burst?

In this project: InMemoryRateLimiterTest demonstrates the boundary burst and the sliding fix.

Q19 [Scenario]. Redis goes down during peak traffic. What happens to your gateways and what should happen?

Requests must not fail because the limiter is down unless policy says so. After the first error mark Redis unavailable for a few seconds (avoid timeout storms), then apply the configured mode: LOCAL = per-instance limiter with limit/replicas so the aggregate approximates the global limit; CLOSED = 503 + Retry-After for quota-critical APIs. Remove Redis from readiness, emit degraded/recovered events, alert on the failure counter. On recovery counters restart empty (brief over-admission) - acceptable and documented.

Likely follow-ups: How do you pick replicas for the divisor? What about burst after recovery?

In this project: ResilientRateLimiterTest (5 tests), troubleshooting exercise 03.

Q20. How do you key rate limits (consumer, route, IP) and what are the pitfalls?

Authenticated traffic: per consumer + route (plan limits). Anonymous: per source IP - but behind proxies you must trust X-Forwarded-For only from your ingress, otherwise clients spoof it and bypass or poison limits. Pitfalls: shared NAT (many users, one IP), IPv6 /64 rotation, unbounded key space (memory), limits per route multiply when a consumer uses many routes (add an overall limit if needed).

Likely follow-ups: How do you stop key-space explosion? Per-consumer global limits?

In this project: GatewayFilter subject key; TRUST_FORWARDED_FOR flag; limiter eviction.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Q21. Local in-memory vs distributed rate limiting: what are the accuracy and scalability trade-offs?

Local limiting is exact per instance, has no network cost and no new failure mode, but the effective global limit is N x limit and varies as the fleet scales. Distributed limiting is globally accurate but adds latency, a dependency, and hot-key/throughput limits in the store. Many systems combine them: local token buckets that periodically borrow from a global budget, or local limiting for abuse protection and distributed limiting for contractual quotas.

Likely follow-ups: How would you reduce Redis load? What error bound can you accept?

In this project: docs/rate-limiting.md local-vs-distributed table.

Q22 [Scenario]. A dependency starts failing and your clients retry. How do you prevent a retry storm?

Retries must be bounded and cheap for the failing system: only idempotent operations, a small attempt cap, exponential backoff with full jitter, an end-to-end deadline so retries cannot exceed the caller's budget, a circuit breaker to stop hammering, and ideally a retry budget (e.g. <=10% of requests). Add load shedding/ bulkheads on the server side and publish client guidance (backoff on 429/503 with Retry-After). Monitor retries vs requests.

Likely follow-ups: Why jitter? What is a retry budget? Who owns the policy: gateway or client?

In this project: ProxyService (safe methods, <=3, jitter, budget), circuit breaker, scenario 7 in docs.

Q23. How does a circuit breaker work and how do you tune it?

States: closed (calls flow, failures counted), open (calls fail fast for a cool-down), half-open (a probe call tests recovery). Tuning: failure criterion (errors/timeouts/slow calls), threshold (consecutive count vs failure rate over a window with a minimum volume), open duration, probe count; per-target not per-route so one bad instance does not eject healthy ones. A consecutive-failure breaker is simple but misses interleaved partial failures; rate-based breakers catch them but need windows and minimum request volumes.

Likely follow-ups: What happens to in-flight requests? How do breakers interact with retries?

In this project: CircuitBreakerTest; ADR-0006 review trigger.

Q24. Bulkhead, rate limit, backpressure: how do they differ?

A rate limit bounds request *rate* per identity (fairness/abuse). A bulkhead bounds *concurrency* to a resource so one dependency cannot consume all threads/connections. Backpressure is the broader principle of pushing overload back to callers (reject early, signal to slow down) rather than queueing unboundedly. In the gateway: per-consumer limiter (rate), per-route semaphore (bulkhead), immediate 503 + Retry-After (backpressure).

Likely follow-ups: Where do queues still make sense? How do you size the bulkhead?

In this project: bulkheadShedsLoadWhenARouteIsSaturated.

Distributed systems

Q25. Per-attempt timeouts versus an end-to-end timeout budget: which and why?

Per-attempt timeouts make worst-case latency attempts x timeout and hide how long the caller actually waited. An end-to-end budget (deadline) bounds total time; retries use only the remaining budget and each attempt gets min(per-attempt cap, remaining). Propagate the deadline downstream (headers/gRPC deadlines) so callees stop work the caller no longer needs. Budgets also cap retry amplification.

Likely follow-ups: How do you propagate deadlines over HTTP? What per-attempt cap would you set?

In this project: ProxyService uses policy.timeoutMs as the whole budget.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Q26. How do you design idempotency for non-idempotent operations?

Client generates a unique key per logical operation; server stores key + request fingerprint + result atomically with the side effect and returns the stored result for repeats; same key with a different payload is rejected; keys expire after the retry horizon; concurrent duplicates are resolved by a unique constraint. For internal event processing use dedupe tables keyed by message id and make handlers commutative where possible.

Likely follow-ups: What if the response was lost? How long do you retain keys?

In this project: IdempotencyService, telemetry (batchId, route, minute).

Q27. Is exactly-once delivery possible?

Not as a transport property across failures; what systems offer is effectively-once processing: at-least-once delivery plus idempotent (or transactional) consumers. Kafka provides transactions and idempotent producers within Kafka; end-to-end with external stores still needs idempotent writes or an outbox/inbox pattern. Design consumers to tolerate duplicates and reordering.

Likely follow-ups: Explain the transactional outbox. What about ordering?

In this project: ADR-0007; telemetry at-least-once with idempotent receiver.

Q28. How do you explain eventual consistency to product owners and design around it?

State the window in plain numbers ("a policy change reaches all gateways within ~35 s worst case"), what users can observe in it, and what emergency tools exist (revoke/disable plus edge rules). Design UI feedback ("pending propagation"), idempotent retries, and monitoring of the lag (snapshot age). Avoid designs that need cross-node transactions when a bounded delay is acceptable.

Likely follow-ups: Which operations cannot be eventually consistent? How do you measure lag?

In this project: docs/policy-propagation.md, gateway_snapshot_age_seconds.

Q29 [Scenario]. The control plane is down for 30 minutes. What breaks and what keeps working?

Traffic keeps flowing on the last-known-good snapshot (memory, plus disk after restarts). Not working: configuration changes, new keys, and revocations/disabled consumers (they keep working until recovery), the console, and telemetry history (buffered then dropped oldest). Alerts: snapshot age, SNAPSHOT_STALE. Mitigations: edge/WAF blocks for urgent revocations, do not restart gateways without a cached snapshot, recover the control plane, verify snapshot age normalises, review whether any revocation was pending.

Likely follow-ups: What if a gateway restarts meanwhile? How would you shorten the revocation gap?

In this project: docs/runbooks/control-plane-outage.md, SnapshotPoller disk cache.

Q30. Kafka or HTTP for telemetry shipping from gateways?

Ask what the requirements are: replay, multiple consumers, durability across consumer downtime, volume. For small, loss-tolerant, per-minute aggregates, batched idempotent HTTP with bounded buffers is far simpler. Kafka (with idempotent consumers and DLQ topics) is justified for per-request analytics, billing-grade data, many consumers or replay. Always quantify operational cost (brokers, partitions, lag monitoring, schema governance) against the requirement.

Likely follow-ups: What would trigger the move? How do you avoid data loss on gateway crash?

In this project: ADR-0007.

Q31. How do you handle dead-letter processing?

Failed messages that will never succeed (poison) or exceed retries go to a DLQ with metadata (error, attempts, correlation id) instead of blocking the queue; alert on DLQ depth; provide tooling to inspect, fix and replay idempotently; classify failures (transient vs permanent) so only permanent ones are dead-lettered. In HTTP-based flows the equivalent is rejecting 4xx batches, counting and dropping them with an alert.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Likely follow-ups: Retention of DLQ? Replay safety?

In this project: Telemetry shipper drops 4xx-rejected batches and counts them.

Q32 [Scenario]. One of three backend instances is slow (p99 8 s) while health checks are green. What do you do?

Health checks verify liveness, not latency, so the instance stays in rotation and drags tail latency. Confirm per-target latency from traces/metrics, then mitigate: eject the instance (manual or outlier detection on latency/error rate), lower its weight, shorten timeouts so slow calls fail over (for safe methods retry another target), and fix root cause (GC, noisy neighbour, bad node). Prevent with outlier ejection, richer health checks and per-target metrics.

Likely follow-ups: Why not just retry? How does the consecutive-failure breaker behave here?

In this project: ADR-0006 limitation (interleaved failures), troubleshooting exercise 01.

Security

Q33 [Scenario]. How would you prevent a gateway from becoming an SSRF vulnerability?

Treat backend URLs as untrusted input: allowlist hosts (exact or suffix), schemes http/https only, no credentials/query/fragment, resolve and reject link-local/metadata/any-local/multicast (and loopback in production), do not follow redirects, re-validate when publishing configuration and again in the data plane before each connection (DNS rebinding), restrict egress with NetworkPolicy or an egress proxy, run with minimal credentials, audit rejections. Never build upstream URLs from request data.

Likely follow-ups: How does DNS rebinding work and what residual risk remains? Allowlist vs blocklist?

In this project: DestinationValidator, DestinationGuard, ADR-0008, tests listed in docs/security.md.

Q34. Difference between RBAC and object-level authorization, and why BOLA is so common.

RBAC answers "may this role call this operation"; object-level authorization answers "may this principal act on *this* resource". Frameworks make RBAC easy (annotations) while object checks need domain knowledge, so they get forgotten, especially on id-based endpoints. Enforce in the service layer next to the data access so every transport (REST, GraphQL, jobs) shares it, scope queries by owner where possible, and write negative tests (other team -> 403).

Likely follow-ups: When return 404 instead of 403? How do you test it at scale?

In this project: AccessGuard; ownerCannotModifyAnotherTeamsApi_BOLA.

Q35. List the JWT validation steps and common mistakes.

Verify signature with a key chosen by configuration (never trust the token header to pick the algorithm or accept `none`), check iss, aud, exp, nbf (with small leeway), optionally typ/azp, check required scopes/roles, handle key rotation via JWKS and kid, keep lifetimes short. Mistakes: accepting unsigned tokens, HS/RS confusion, skipping aud, long lifetimes, trusting claims before verification, storing tokens in localStorage, no revocation story.

Likely follow-ups: How do you handle revocation? Clock skew?

In this project: Gateway/CP tests: expired, wrong issuer, forged signature, future nbf, alg:none.

Q36. Design secure API key storage and rotation.

Generate 256-bit random secrets, show the plaintext once, store only a hash (SHA-256 is sufficient for high-entropy secrets; slow hashes are for passwords), use a non-secret prefix for lookup, compare in constant time even for unknown prefixes, add expiry and per-consumer scoping, support multiple active keys for rotation (issue new, switch, revoke old), audit issue/revoke, and never log or return keys again.

Likely follow-ups: Why not bcrypt? How do you detect leaked keys?

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

In this project: ConsumerService, Authenticator, apiKeyIsShownOnce... test.

Q37. Explain CORS and CSRF and decide for this API.

CORS is a browser mechanism relaxing the same-origin policy for cross-origin reads; it protects nothing server-side. CSRF abuses ambient credentials (cookies) so the browser sends authenticated requests from attacker pages. With bearer tokens in the Authorization header there are no ambient credentials, so CSRF protection is unnecessary; CORS still needs an explicit origin allowlist (or same-origin proxying). If you add cookie auth, enable CSRF tokens and SameSite.

Likely follow-ups: What about BFF with cookies? Why not wildcard CORS with credentials?

In this project: SecurityConfig comments; nginx same-origin proxy.

Q38 [Scenario]. An API key was found in a public repository. What do you do?

Treat as compromised: revoke immediately (propagation takes seconds - add edge blocks if needed), identify the consumer and issue a replacement, inspect audit and access logs for use during the exposure window, rotate related secrets, remove the secret from history (and assume it was copied), notify the owner and security, and add prevention: secret scanning pre-commit/CI, short expiry, per-consumer keys, alerts on anomalous use. Document in a post-incident review.

Likely follow-ups: How do you find usage during the window? How do you avoid revocation delay?

In this project: Revoke path + audit; runbook incident-response (credential leak).

Q39. How would you prioritise the OWASP API Top 10 for a gateway?

By exposure and impact for this system: BOLA/authorization, authentication, unrestricted resource consumption (limits), SSRF, security misconfiguration, then property-level exposure and inventory. Build each into design (deny by default, object checks, bounded everything, destination allowlist) and verify with negative tests and scanning rather than checklists alone.

Likely follow-ups: Where do WAFs help or not?

In this project: docs/security.md OWASP mapping.

Q40. How do you threat-model a gateway?

Draw the data flows and trust boundaries (consumer -> ingress -> gateway -> backend; operator -> control plane -> DB; gateway <-> control plane), enumerate threats per boundary (STRIDE), rate by likelihood/impact, map to controls and tests, record residual risks with owners, and revisit when inputs or boundaries change (new backend type, new auth mode). Keep the model in the repository next to the code.

Likely follow-ups: Who should review? How often?

In this project: Threat table in docs/security.md.

Kubernetes

Q41 [Scenario]. Design probes for a JVM gateway whose readiness depends on configuration but not on Redis.

Startup probe on liveness with a generous failure threshold covers slow starts; liveness is a cheap in-process check (never depends on external systems); readiness includes "configuration snapshot loaded" and excludes Redis so a Redis outage degrades limiting without removing pods. Keep probes unauthenticated, light, with timeouts shorter than the period.

Likely follow-ups: What happens if readiness depends on the control plane? Why not use a TCP probe?

In this project: Actuator groups + SnapshotHealthIndicator, manifests.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Q42 [Scenario]. How do you achieve zero-downtime deployments and graceful shutdown?

RollingUpdate with maxUnavailable 0 and surge, readiness gating, preStop sleep so endpoints deregister before SIGTERM, application graceful shutdown that stops accepting new requests and finishes in-flight within a bound, grace period larger than preStop + drain, PDBs during node drains, and backward-compatible schema/config so old and new pods coexist. Verify with a load test during a rollout.

Likely follow-ups: Why a sleep in preStop? What about long-lived connections?

In this project: Manifests: preStop 10 s + 25 s drain < 40 s.

Q43. How do you set requests and limits for JVM workloads?

Requests reflect typical usage (scheduling and HPA denominators); limits protect the node. Memory: limit >= heap + ~25-30% non-heap, avoid limit < observed peak RSS, set MaxRAMPercentage. CPU: limits can throttle JVM/GC threads - consider no CPU limit or generous limits and rely on requests/HPA; always measure throttling (`container_cpu_cfs_throttled_*`).

Likely follow-ups: What is Guaranteed QoS? Why can CPU limits hurt latency?

In this project: Manifests and troubleshooting 09.

Q44 [Scenario]. Your gateway HPA flaps between 3 and 12 replicas. What do you check?

Metric choice (CPU lags bursty traffic; consider RPS or concurrency), target utilisation too low, scale-down stabilisation window too short, readiness delay making new pods appear late, startup time, per-pod CPU requests unrealistic, noisy metrics. Add behavior policies (fast scale-up, slow scale-down), warm-up, and load-test the scaling curve.

Likely follow-ups: Custom metrics for HPA? What about KEDA?

In this project: HPA behavior block in manifests.

Q45. How do NetworkPolicies work and how would you roll them out safely?

They are additive allow-lists applied by the CNI per pod selector; selecting a pod with policyTypes makes it default-deny for those directions. Roll out by mapping flows (service graph, flow logs), adding allow policies first, then default-deny namespace by namespace, testing connectivity (and DNS egress!), and keeping a rollback. Remember egress to DNS and observability backends.

Likely follow-ups: Does every CNI enforce them? How do you test?

In this project: infrastructure/k8s/40-*.yaml; backends only reachable from the gateway.

Q46 [Scenario]. A Helm upgrade leaves pods NotReady. How do you respond?

If `--atomic` is used Helm rolls back automatically; otherwise `helm rollback` to the previous revision, then investigate with `kubectl describe`, events, logs and probe configuration. Typical causes: missing secret key, changed config, migration lock, probe path wrong. Add pre-deploy validation (`helm template | kubeconform`), staging runs, and readiness that tests real dependencies.

Likely follow-ups: Hooks vs init containers for migrations? How do you handle DB changes during rollback?

In this project: docs/kubernetes-and-helm.md flow; CI kubernetes job.

Q47 [Scenario]. Pods are CrashLoopBackOff. Walk through your investigation.

`kubectl get pods` and `describe` (last state reason/exit code: 137 OOMKilled, 1 app error), `logs --previous`, events, probe failures, config/secret existence (keys only, never values), resource usage vs limits, image/arch, read-only filesystem write attempts, network policy blocks to dependencies. Fix the cause (limits, config), not the symptom (raising probe thresholds).

Likely follow-ups: How do you debug a distroless image? Ephemeral containers?

In this project: Exercise 09.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Q48. How should secrets be handled in Kubernetes?

Do not commit them; reference Secrets created out-of-band or synced from a secret manager (External Secrets, CSI driver), enable encryption at rest, restrict RBAC to read secrets, prefer mounted files over env vars where `exec`/crash dumps could leak them, rotate regularly, and use workload identity instead of static cloud credentials.

Likely follow-ups: Are Kubernetes Secrets encrypted? Sealed Secrets vs External Secrets?

In this project: Helm existingSecret; k8s secret.example with invalid values.

Cloud architecture

Q49 [Scenario]. Map this platform to AWS. What would you use and what would you check first?

EKS for gateway/control plane/UI (managed node groups or Karpenter), Aurora/RDS MySQL Multi-AZ for the control plane database, ElastiCache Redis (replica + Multi-AZ) for limiter counters, ALB with ACM certificates (or NGINX + cert-manager) at the edge with WAF/Shield, Secrets Manager via External Secrets, IRSA for pod permissions, ECR with scan-on-push, Amazon Managed Prometheus/Grafana and an OpenTelemetry Collector (to X-Ray/Jaeger). First check: failure behaviour of managed failover (Redis, DB) against your timeouts, plus quotas and subnet sizing.

Likely follow-ups: Where do you place the gateway - public vs private subnets? How do you secure egress?

In this project: docs/kubernetes-and-helm.md cloud mapping table (conceptual - not deployed).

Q50. How would you design for multiple AZs and regions?

Spread gateway and control-plane replicas across AZs (topology spread), use Multi-AZ managed data stores, keep gateways independent of cross-AZ calls on the hot path. Multi-region: gateways per region with regional snapshot caches, a single-writer control plane region with replicas, DNS/health-based failover, and clear RPO/RTO. Active-active control planes need conflict-free design for approvals and revocations - usually not worth it early.

Likely follow-ups: What is your RTO for the control plane? How do you fail back?

In this project: Gateways serve last-known-good during control-plane loss (fail-static).

Q51. Managed services versus self-hosting MySQL and Redis?

Managed services give backups, patching, HA and monitoring at a cost and with provider-specific limits; self-hosting offers control but requires expertise and on-call. Prefer managed unless a requirement forbids it, and design the application to tolerate failover (timeouts, retries for idempotent DB calls, Redis fail modes). Test failover behaviour - managed does not mean no failures.

Likely follow-ups: How do you test a managed failover? What are connection storm risks?

In this project: Compose/K8s demo stores are labelled demo-only.

Q52. How do you control the cost of observability?

Cost drivers are metric series, trace volume and log volume. Control cardinality (bounded labels), sample traces (head, then tail sampling for errors/slow), log at INFO with no payloads, set retention per tier, aggregate before shipping (per-minute buckets), downsample/recording rules, and review the top-series list regularly. Define what questions each signal must answer to justify its cost.

Likely follow-ups: How do you decide sampling rates? What do you never sample?

In this project: docs/observability.md cost section; telemetry buckets; sampling config.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Q53. How do workloads get cloud permissions without static keys?

Use workload identity: IRSA on EKS, Workload Identity on GKE/AKS, mapping a Kubernetes ServiceAccount to a cloud role with least privilege; short-lived credentials issued by the platform, no keys in images/env; CI uses OIDC federation to assume a deploy role. Audit with cloud trail/logs and review permissions regularly.

Likely follow-ups: How do you scope a role to one namespace/service? What about local dev?

In this project: Manifests use per-workload ServiceAccounts; docs list IRSA.

Q54. Where do CDN and WAF fit?

In front of the ingress for TLS offload, caching of cacheable GETs (carefully for authenticated APIs), bot/volumetric protection, geo/IP rules and virtual patching. They complement, not replace, gateway rate limiting and authentication; ensure the origin only accepts traffic from the edge and that client IPs are forwarded safely (XFF trust).

Likely follow-ups: What should never be cached? How do you stop origin bypass?

In this project: Ingress annotations and TRUST_FORWARDED_FOR note.

Q55 [Scenario]. Define RTO/RPO for the control plane and design the backup/DR plan.

Start from business impact: control-plane loss does not stop traffic, so RTO can be hours but revocation capability is the constraint; RPO is determined by DB replication/backups (e.g. minutes with Multi-AZ + PITR). Plan: automated backups with PITR, cross-region snapshot copies, tested restore runbooks, infrastructure as code to recreate the cluster, and game days. Gateways keep snapshots; after restore verify audit continuity and snapshot versions.

Likely follow-ups: How often do you test restores? What if the audit table is corrupted?

In this project: Gap noted in production-readiness assessment (no restore drill).

Q56 [Scenario]. You are migrating an on-premises gateway to this platform. How do you de-risk it?

Inventory APIs and consumers, import them into the registry, run the new gateway in shadow/mirror mode and compare responses and latency, migrate consumers gradually behind DNS/weighted routing, keep identical policies initially, monitor SLOs per API, keep an instant rollback path, and freeze unrelated changes. Migrate low-risk APIs first and document differences (headers, error formats).

Likely follow-ups: How do you handle consumer credentials? What about long-lived connections?

In this project: Registry + versioned policies + per-route metrics support staged migration.

Observability

Q57 [Scenario]. Prometheus memory is growing and queries are slow. How do you reduce telemetry cardinality?

Find the offenders: TSDB status page and `topk(10, count by (__name__)({__name__=~".+"}))`, then per-label counts. Remove or bucket unbounded labels (user id, raw path, IP), replace path with the route template, drop unused metrics via relabeling, reduce histogram buckets, use recording rules for expensive queries, and move per-entity detail to logs/traces. Add guardrails: label allowlists in code review and a CI test that random inputs do not create series.

Likely follow-ups: What is the memory cost per series? How do you find a runaway label in production?

In this project: Route label bounded; test prometheusExposesRouteLevelMetricsWithBoundedLabels.

Q58. Metrics, logs and traces: when do you use which?

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Metrics for detection and trends (cheap, aggregated); traces for causality and latency breakdown across services (sampled); logs for detailed events and forensic context. Workflow: alert from metrics -> narrow by labels -> pick a trace/correlation id -> read logs of the failing hop. Correlate them with trace ids in logs and exemplars linking metrics to traces.

Likely follow-ups: What are exemplars? How do you keep logs affordable?

In this project: Log pattern with traceId/cid; observability doc.

Q59. Why can you not average percentiles, and how do you compute them correctly?

A percentile of percentiles has no statistical meaning. Record latency in histograms with fixed buckets, sum bucket counts across instances/time, then apply histogram_quantile to the aggregated buckets. Accuracy depends on bucket boundaries around the quantile (put buckets near the SLO), and low request counts make p99 noisy.

Likely follow-ups: How do you choose buckets? What are native histograms?

In this project: gateway_requests_seconds histogram with SLO buckets 50 ms - 5 s.

Q60. Head sampling versus tail sampling?

Head sampling decides at the start (cheap, simple, random) and can drop the rare error/slow traces you most want. Tail sampling decides after the trace completes (keep all errors and slow traces plus a small random share) but needs a Collector that buffers spans and consistent routing by trace id. Combine: low head rate plus tail policies at the Collector; keep sampling decisions propagated so partial traces are avoided.

Likely follow-ups: What does tail sampling cost? How do you debug missing spans?

In this project: TRACE_SAMPLING config; tail sampling listed as production upgrade.

Q61. Correlation id versus trace id?

A trace id is generated by the tracing system with a specific format and may be sampled out; a correlation id is a business/support-friendly identifier that survives sampling, is returned to the client and appears in logs and audit. Use both: propagate W3C traceparent for tracing and X-Correlation-Id for humans; validate inbound values to prevent log injection.

Likely follow-ups: Can they be the same? How do you secure the header?

In this project: CorrelationFilter, GatewayFilter, ProxyService.

Q62. How do you alert on SLOs?

Define SLIs (good/valid requests), SLO targets and error budgets, then alert on **burn rate** over multiple windows (e.g. 14x over 5 min and 1 h for paging, 3x over 6 h for tickets) to catch fast and slow burns with low noise. Alert on symptoms users feel; keep cause-based alerts as dashboards. Every page links to a runbook.

Likely follow-ups: Why multi-window? How do you pick targets?

In this project: alerts.yml (symptom-first) with runbook links.

Q63 [Scenario]. Your 504 rate doubled in the last 20 minutes. How do you investigate?

Confirm scope (which routes, since when), check recent changes (policy/timeout, deploys, target removals) via the incident view and audit, split 504 vs 502/503, compare policy timeoutMs with backend p95/p99, check target health/breaker events, gateway retries (hidden latency), backend saturation (CPU, GC, DB pool), network/DNS. Use a trace from a failing correlation id to see where the time goes. Mitigate first: revert a recent change, shift traffic from a slow target, raise timeout only with evidence.

Likely follow-ups: How do retries affect the 504 count? What if only one target is affected?

In this project: Exercises 01/02; BACKEND_TIMEOUTS rule.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Q64 [Scenario]. Traces for a request are split into separate traces at the gateway. What do you check?

Whether the gateway injects traceparent on the proxied call (and does not strip it), whether the backend extracts it with the same propagator (W3C vs B3), sampling consistency, header handling by intermediaries, and async hand-offs losing context. Inspect the headers the backend actually receives with an echo endpoint and compare trace ids in both logs.

Likely follow-ups: How do you test propagation automatically?

In this project: Exercise 08; ProxyService.injectTrace.

Performance engineering

Q65 [Scenario]. A release meets all functional requirements but fails performance targets. What do you do?

Do not ship on hope. Quantify the gap against the SLO (p95/p99, throughput, resource use) and reproduce it with a repeatable test. Profile to find the cause (allocation, blocking, N+1, locks, extra hops), fix or roll back the offending change, re-measure, and decide with the product owner: delay, ship behind a flag/canary with limits and rollback triggers, or accept a documented exception with an expiry. Communicate numbers and risk plainly; add a performance gate so it is caught earlier next time.

Likely follow-ups: Who decides to accept the risk? How do you canary on performance?

In this project: Release readiness and performance checklists; stakeholder risk template.

Q66. How do you design a trustworthy load test?

Use an open model (constant arrival rate) so slow responses do not reduce offered load (coordinated omission), warm up, run long enough for steady state, keep generator and SUT on separate machines, use realistic data/mix, include failure scenarios, monitor resources and percentiles, repeat runs and report spread, and compare with a baseline. State limits honestly - single laptop numbers are relative only.

Likely follow-ups: What is coordinated omission? How many runs do you need?

In this project: docs/performance.md caveats; LoadProbe vs k6 scripts.

Q67. The gateway adds latency. How do you reduce it?

Measure first: split gateway overhead vs backend time in traces. Then remove per-request work (DNS, regex, object creation), avoid extra copies/buffering, reuse connections (keep-alive pool), cache immutable decisions, make limiter operations O(1), batch telemetry, and use HTTP/2 or connection coalescing where beneficial. Accept costs that buy safety (auth, limits) and optimise their implementation, not their existence.

Likely follow-ups: What is the theoretical minimum? When is a second hop unavoidable?

In this project: Measured gateway vs direct and rejection-path results.

Q68 [Scenario]. DB connection pool exhaustion pages you at 2 am. How do you respond and prevent it?

Mitigate: identify long-running queries/transactions (processlist, innodb_trx), kill the offender, roll back the recent release, temporarily raise pool only if the DB has headroom. Diagnose with Hikari usage/pending metrics and thread dumps. Prevent: paginate and index, short read-only transactions, no remote calls in transactions, statement timeouts, fail-fast pool timeouts, alerts on pending > 0, and load tests with realistic data.

Likely follow-ups: Why not simply increase the pool? How do you find the query?

In this project: Exercise 04; HikariPoolSaturated alert; QueryPlanIT.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Q69 [Scenario]. A query that was fast is now slow in production. What is your method?

Capture the exact statement and parameters from the slow log, run EXPLAIN ANALYZE, compare plans with the previous version/statistics, check data growth, missing/changed indexes, parameter skew and lock waits. Fix via index/query rewrite (migration with expand/contract), refresh statistics, add a plan assertion test for the hot queries, and monitor.

Likely follow-ups: What does "Using filesort" imply? Keyset vs offset pagination?

In this project: V2 index migration, QueryPlanIT.

Q70. How do you choose and invalidate caches?

Cache only after measuring a need; define the consistency rule (how stale is acceptable), key design, bounded size/TTL and the invalidation trigger (after-commit event, version in key). Prefer immutable snapshots swapped atomically over mutable caches. Beware cache stampedes and stale-on-error behaviour.

Likely follow-ups: Write-through vs cache-aside? How do you test staleness?

In this project: Snapshot cache invalidated after commit + 30 s timer; no Redis in front of MySQL (ADR-0004).

Q71. How do you report benchmark results honestly?

State the method (tool, open/closed model), environment, versions, warm-up, duration, concurrency, repeat count and spread; show percentiles and error counts, not just averages; note confounders (shared CPU, noisy neighbours) and what was not measured; never present them as capacity. Keep raw outputs with the report.

Likely follow-ups: What would make you reject a benchmark? How do you compare two versions?

In this project: docs/performance.md and docs/validation/perf-*.txt.

Q72. How do you do capacity planning for the gateway fleet?

Load-test one pod to find the CPU-bound throughput at target latency and error rate, apply a safety margin (e.g. 60% utilisation), multiply by peak demand with growth, account for N+1/AZ-loss headroom and failover, set HPA targets and min replicas from that, and verify with a ramp test and chaos (kill a zone). Re-measure after significant changes.

Likely follow-ups: Why 60%? How do you include headroom for retries?

In this project: HPA targets in manifests; method only - no capacity claimed.

Testing

Q73. What is your test strategy for a gateway?

Unit-test pure logic (validators, algorithms, route table), integration-test the real pipeline with a real HTTP backend (routing, auth matrix, limits, timeouts, retries, breaker, bulkhead, bad config), contract-test configuration formats, run Testcontainers for Redis/MySQL, add failure/recovery and concurrency tests, security regression tests, and a small end-to-end smoke suite plus performance tests. Keep tests deterministic with fake clocks and unique data.

Likely follow-ups: How do you prevent over-mocking? What belongs in e2e?

In this project: docs/testing-strategy.md; GatewayRoutingIT.

Q74. How do you test concurrent code?

Design for testability (injectable clocks, small critical sections), then use many threads released by a latch against shared state and assert invariants (exactly N admitted, unique versions), repeat runs, use jcstress/stress options for hard cases, and rely on database constraints as ground truth. Avoid sleeps; assert on outcomes not timing.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Likely follow-ups: How do you find flaky races? When is a lock-free structure worth it?

In this project: InMemoryRateLimiterTest, concurrentPolicySubmissions test.

Q75 [Scenario]. How do you test failure and recovery behaviour such as Redis outages?

Replace the dependency with a controllable fake for unit tests (throw, delay, recover) and assert modes and back-off, then use Testcontainers/chaos tooling to stop and start the real dependency in integration tests, asserting availability, headers, events and recovery. In staging run game days (kill Redis during load) and check bounded over-admission.

Likely follow-ups: How do you simulate slow rather than down? What do you assert after recovery?

In this project: ResilientRateLimiterTest; real Redis outage test is a documented gap.

Q76. How do you contract-test the control plane / gateway boundary?

Define the JSON contract in one document, test that the producer emits it (control-plane tests parse the snapshot) and that the consumer tolerates it (gateway ignores unknown fields, validates required ones), keep fixtures versioned, and run N and N-1 combinations in CI. Consumer-driven contracts (Pact) formalise this when many consumers exist.

Likely follow-ups: How do you evolve a field safely?

In this project: docs/api-contracts.md; ignoreUnknown on the gateway Snapshot.

Q77. Testcontainers versus H2: trade-offs?

H2 is fast and needs no Docker but differs in dialect, locking, index behaviour and SQL features - good for most logic, risky for migrations and query plans. Testcontainers gives real engines (MySQL, Redis) at the cost of speed and Docker availability. Use H2 for the broad suite and a small Testcontainers set for migrations, plans, Lua scripts and engine-specific behaviour (auto-skipped without Docker - and report that skip).

Likely follow-ups: How do you keep CI time down? What if Docker is unavailable?

In this project: MySqlMigrationIT and RedisRateLimiterIT skipped here; documented.

Q78. How do you deal with flaky tests?

Treat flakiness as a bug: quarantine only temporarily with an owner, find the cause (time, ordering, shared state, async), fix via fake clocks, unique data, explicit waits on conditions, and retries only for infrastructure. Track flake rates; never normalise ignoring red builds.

Likely follow-ups: When is a retry acceptable? How do you detect order dependence?

In this project: Unique identifiers per test; random Jasmine order enabled.

Q79. What belongs in a security regression suite?

Authentication negatives (missing/expired/forged tokens), authorization negatives (wrong role, other tenant, disabled entity), input abuse (oversized, malformed, injection strings, traversal), SSRF attack strings, secret exposure checks (no keys in responses/audit), header spoofing, rate-limit and error-message behaviour. Each fixed vulnerability gets a test.

Likely follow-ups: How do you keep it maintainable? How does it relate to pen testing?

In this project: See security section of docs/testing-strategy.md.

Q80. How do you test an Angular app effectively?

Unit-test services/interceptors/guards with HttpTestingController, component-test through the DOM (rendering states, validation messages, role-based visibility) without reaching into private state more than necessary, and keep a small Playwright suite for cross-page flows against a real backend. Lint in CI, avoid sleeps, test accessibility basics (labels/roles).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Likely follow-ups: What do you mock? How do you test lazy routes?

In this project: 27 Karma specs + 8 Playwright tests.

AI-assisted development

Q81 [Scenario]. How would you structure a specification-driven implementation using an AI coding assistant?

Write (or have the assistant draft and a human approve) a spec with objective, acceptance criteria, API contract, security, failure behaviour, observability, performance expectations, test scenarios and rollout. Derive a design and dependency-ordered tasks; run each task with a small context bundle (project context + relevant modules + exact files). Require tests first or alongside, run real commands and attach outputs, then independent review (security/performance) and human approval for sensitive changes. Keep specs and ADRs in the repo so context survives sessions.

Likely follow-ups: Where do humans approve? How do you handle spec changes mid-flight?

In this project: ai-engineering/spec-driven-workflow.md; specs/001-ssrf-destination-guard and specs/feature-quota-policy.md.

Q82 [Scenario]. How do you evaluate AI-generated code for correctness and security?

Read it as an untrusted PR: does it match the spec and invariants; are APIs real (compile/run); check concurrency, authorization, input handling, resource bounds, error paths; run the tests and add negative/adversarial ones; mutate the code to confirm tests fail; run static analysis and dependency scans; review diffs for silent behaviour changes and invented configuration. Use the review modules and a checklist; require provenance and evidence in the PR.

Likely follow-ups: What do you do when the assistant writes the tests too? How do you review a 3k-line diff?

In this project: review-examples/examples.md (12 cases tied to real tests).

Q83 [Scenario]. How would you patch a vulnerable dependency without introducing regressions?

Confirm the advisory and its reachability, pick the smallest fixed version, add regression tests for behaviour you rely on (and for the vulnerable behaviour if testable), check the dependency tree and release notes, run full unit/integration/e2e plus a quick performance comparison for hot-path libraries, deploy via canary with a rollback ready. If no fix exists, mitigate or document an accepted risk with owner and expiry.

Likely follow-ups: What if the fix needs a major version bump? How do you test transitive changes?

In this project: java-jwt 4.4.0 -> 4.6.2 evidence; npm audit acceptance (docs/governance/vulnerability-remediation.md).

Q84. What is prompt injection and how does it affect coding assistants?

Untrusted text (web pages, issues, dependency READMEs, code comments, tool output) can contain instructions that the model may follow, causing data exfiltration or destructive actions. Mitigate with least-privilege tools, no secrets in the workspace, human approval for commands with side effects, sandboxed execution, treating tool output as data, restricting network access, and reviewing proposed commands. Never let assistants run untrusted code or fetch-and-execute.

Likely follow-ups: How do you handle instructions found in a downloaded file? What permissions would you deny?

In this project: AGENTS.md prohibitions; rule: external input is untrusted.

Q85. What must never be given to an AI assistant?

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Secrets, credentials, private keys, tokens, production data, customer PII/PHI, unreleased vulnerabilities details beyond need, and proprietary data not approved for the tool. Use synthetic data, redact logs, scope repository access, and keep instruction files free of sensitive content; follow the organisation's data policies and tool agreements.

Likely follow-ups: How do you prevent accidental pastes? How do you handle logs for debugging?

In this project: Rule 1 in ai-engineering/README.md; dummy credentials only.

Q86. How would you measure the impact of AI assistance on delivery?

Define outcome metrics (lead time, review time, defect escape rate, change failure rate, rework, test coverage of changed code), compare comparable work with and without assistance over enough samples, include quality (post-merge defects, incidents) not just speed, and survey developers. Avoid vanity metrics (lines generated). No measurements exist for this repository and none are claimed.

Likely follow-ups: What confounders exist? How long should the pilot be?

In this project: Honesty statements throughout the workspace.

Q87. When should you not use an AI assistant?

When the work is security-critical and you cannot verify it, when context is confidential or not permitted, when you do not understand the domain well enough to review the output, for tasks where a deterministic tool exists (formatting, codegen, migrations by tool), and when the verification cost exceeds writing it yourself. Use it for exploration, boilerplate, tests, refactoring with strong tests, documentation drafts.

Likely follow-ups: How do you stay skilled? What is your review budget?

In this project: Human approval gates in the workflow.

Q88 [Scenario]. During an incident an assistant proposes a root cause. How do you use it safely?

Separate verified facts from hypotheses: require it to cite evidence (metrics, logs, trace ids) and list what would confirm or refute each hypothesis; verify with read-only commands; never let it execute restarts or reverts without human approval; keep deterministic incident rules as the primary evidence source and label AI output as a hypothesis in the UI and report.

Likely follow-ups: What if it is confidently wrong? How do you log its involvement?

In this project: incident-troubleshooting module; incident view has no AI content.

Context engineering

Q89 [Scenario]. How would you optimise repository context and token usage without losing important requirements?

Keep a short always-loaded context (purpose, boundaries, commands, invariants, DoD) and load specialised modules per task; use a repository map and targeted search instead of whole-tree dumps; read the smallest relevant ranges and tests; capture decisions in ADRs and rules files; hand off with summaries and paths, not transcripts. Guard requirements by keeping them in numbered acceptance criteria referenced by tasks, and measure outcomes (pass rate, tool calls, tokens) before and after to ensure quality did not drop.

Likely follow-ups: How do you decide what stays always-loaded? How do you avoid stale instructions?

In this project: ai-engineering/context-engineering-guide.md; before/after examples; no token savings claimed.

Q90 [Scenario]. How do you detect that an agent is missing necessary context?

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Signals: invented classes/properties, no file citations, questions answered by repository contents, violations of stated invariants, hedged claims, repeated failed attempts. Ask it to list the files it read, assumptions and unknowns; then supply the missing artifact or rule. Add a standing instruction to list unknowns before coding and to cite paths.

Likely follow-ups: What if the context window is full? How do you make unknowns explicit?

In this project: Context guide section on detecting missing context.

Q91. How do you design repository instruction files?

Short, stable, and actionable: rules and pointers rather than copies of docs; tool-neutral core with thin adapters; no secrets; commands that actually work; "inspect before change" lists; explicit prohibitions requiring human approval. Review them like code and update when reviews reveal repeated mistakes.

Likely follow-ups: How long should they be? How do you test them?

In this project: CLAUDE.md, AGENTS.md, copilot-instructions, ai-engineering/*.md.

Q92. What makes a good handoff note between agent tasks?

Goal, state, paths changed, decisions and why, contracts touched, evidence (commands and real results), skipped/unverified items, open questions, what the next role should load and must not change, and approvals needed. It lets a fresh session continue without the transcript.

Likely follow-ups: How do you stop notes from becoming noisy? Who reads them?

In this project: multi-agent/handoffs/_template.md.

Q93. What are the risks of summarising conversations for context?

Summaries drop caveats, compress uncertainty into certainty, and can persist mistakes; decisions without rationale get re-litigated. Mitigate by summarising into structured artifacts (ADRs, specs, handoffs) with evidence links, keeping unresolved questions explicit, and re-verifying critical facts against the code.

Likely follow-ups: How do you handle contradictory summaries?

In this project: ADR review triggers; UNVERIFIED markers.

Q94. How do you measure token savings credibly?

Run identical tasks with the old and new context on the same model in fresh sessions, record tokens, tool calls, wall time and correctness (tests/review) from the tool's usage report, compare medians over several task types, and include pass rate; also compute the fixed per-turn overhead of the always-loaded files. Report model versions and dates.

Likely follow-ups: How many samples? What if savings reduce quality?

In this project: Measurement method in the context guide (no numbers invented).

Q95. How do you prevent instruction drift and staleness?

Treat instructions as code: owners, review in PRs, tests where possible (commands run in CI), a repo map regenerated when structure changes, expiry/"last verified" dates for volatile facts, and removal of rules that duplicate or conflict. Periodically replay representative tasks to see if behaviour matches the rules.

Likely follow-ups: How do you detect conflicts between modules?

In this project: CODEOWNERS covers ai-engineering/.

Multi-agent workflows

Q96 [Scenario]. How would you prevent multiple AI agents from introducing conflicting architecture decisions?

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Make architecture a frozen input: ADRs and architecture rules are owned by an architecture role and changed only through human-approved ADRs; agents propose, never edit them. Define shared contracts first, give each agent file ownership, sequence dependent tasks, require handoffs with decisions listed, detect drift with contract tests and CI, and have an independent reviewer check invariants. Conflicts escalate to the architecture role and then a human.

Likely follow-ups: What if two valid designs conflict? How do you detect silent contract drift?

In this project: ai-engineering/multi-agent/README.md; architecture-rules.md.

Q97 [Scenario]. Three agents need to touch overlapping files. How do you coordinate?

Avoid overlap by decomposition: shared types go through the contract docs first; assign single-writer ownership per file area; run dependent tasks sequentially and independent tasks in parallel on disjoint files (or separate worktrees/branches); rebase and re-run gates on merge; if semantic conflicts appear, stop and escalate to the architecture role.

Likely follow-ups: How do you handle migrations? Branch per agent?

In this project: Single-writer rule for migrations in the protocol.

Q98 [Scenario]. Can an implementation agent approve its own security-sensitive change?

No. High-risk changes (auth, SSRF controls, migrations, infra, hot-path dependency upgrades) need an independent reviewer (another role or human) and explicit human approval at the security gate; self-approval defeats the purpose because the author shares the same blind spots and context. The reviewer sees the diff and the spec, not the author's reasoning.

Likely follow-ups: What counts as independent? Who can override a blocker?

In this project: Review gates G6/G9.

Q99. What are the main failure modes of multi-agent systems?

Conflicting assumptions, context bloat from long transcripts, duplicated or contradictory edits, silent contract changes, agents agreeing with each other (shared hallucinations), unclear ownership, runaway cost and unbounded loops, weak evidence ("tests pass" without output), and security risks from broad tool permissions. Controls: contracts, ownership, gates, evidence rules, budgets, least privilege, and human checkpoints.

Likely follow-ups: How do you cap cost? How do you detect shared hallucination?

In this project: multi-agent/README principles.

Q100. How should agent permissions and sandboxing be configured?

Least privilege per role: read-only for reviewers, write limited to owned paths, no network or secrets by default, deny destructive commands and pushes without approval, run in disposable workspaces, log tool calls, and review commands for injection. Different roles should not share credentials.

Likely follow-ups: How do you enforce path ownership technically? What about CI tokens?

In this project: AGENTS.md prohibitions; CODEOWNERS.

Q101. What evidence should an agent provide with each handoff?

Exact commands run, exit status, counts (passed/failed/skipped), links to reports, list of skipped or blocked checks with reasons, diff statistics and files touched, plus claims explicitly marked UNVERIFIED when not checked. Evidence must be reproducible by the reviewer.

Likely follow-ups: How do you stop fabricated evidence? What is acceptable proof of a test run?

In this project: Handoff template evidence table.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Q102. When is a single agent better than several?

For small, tightly coupled changes the coordination overhead, context duplication and merge risk outweigh parallelism; a single agent with a good spec and checklists is simpler and cheaper. Use multiple roles when work is separable, needs independent review, or benefits from distinct expertise - and keep the number of agents small.

Likely follow-ups: How do you decide? What would you measure?

In this project: This repository was built by one assistant session; the multi-agent protocol is designed, not executed.

Troubleshooting scenarios

Q103 [Scenario]. Intermittent 502 and 504 from the gateway: how do you proceed?

Split 502 vs 504 and per target; check health/breaker events, backend latency against timeoutMs, keep-alive/connection reset patterns, and retries amplifying load; compare a failing trace to a good one. Mitigate by removing or de-weighting the bad target and adjusting timeout from measured p99; add outlier ejection for partial failures.

Likely follow-ups: Why can a consecutive-failure breaker miss this?

In this project: docs/troubleshooting/01.

Q104 [Scenario]. Latency rose after a release; what is your approach?

Annotate the release on dashboards, compare p50/p95/p99 before/after, split gateway vs backend in traces, diff changes (code, config, policy), check resources (CPU throttling, GC), look for hidden retries or DNS/serialisation costs, and roll back if the impact is user-visible while you profile with JFR.

Likely follow-ups: What if only p99 moved? How do you attribute it to a commit?

In this project: docs/troubleshooting/02.

Q105 [Scenario]. Rate limiting seems random for one consumer. Why?

Likely mode flapping between Redis and local fallback, replica-count mismatch in the fallback divisor, Redis eviction of counters, XFF/IP keying differences for anonymous traffic, or the fixed-window boundary effect. Inspect X-RateLimit headers, degraded events, Redis stats and replica counts; fix config and consider CLOSED mode for strict quotas.

Likely follow-ups: How can headers tell you which limiter answered?

In this project: docs/troubleshooting/03.

Q106 [Scenario]. A policy change broke a partner's traffic. What do you do?

Revert to the previous policy version (creates a new version, audited) - propagation takes seconds - while communicating the window; use the audit trail to see exactly what changed and who approved; after recovery, improve review by showing effective per-second rates and diffs, and add alerts on 429/403 jumps after approvals.

Likely follow-ups: How do you shorten propagation for emergencies?

In this project: docs/troubleshooting/07; revert endpoint.

Q107 [Scenario]. Pods keep restarting after a config change. Walk me through it.

Describe the pod (exit code, last state), read previous logs, check probes vs startup time, secrets/config keys exist, memory vs limit, read-only filesystem write attempts, network policy blocks; roll back the release if needed, fix the cause, keep startup probes sensible.

Likely follow-ups: How do you debug without a shell in the image?

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

In this project: docs/troubleshooting/09.

Q108 [Scenario]. A library upgrade caused authentication failures in staging. How do you find and fix it?

Bisect between versions with tests, diff the dependency tree and release notes, reproduce with the failing token (clock skew, nbf/exp validation, issuer/typ handling), pin or roll back, then add boundary tests for the behaviour and re-upgrade with a canary. Keep security libraries behind a thin adapter you test.

Likely follow-ups: What would you add to the test fixtures? How do you avoid this next time?

In this project: docs/troubleshooting/10; added nbf and alg:none tests.

Q109 [Scenario]. Gateway memory grows for days. Method?

Heap-after-GC trend, class histogram diff, heap dump with MAT, inspect per-key structures (limiter keys), buffers and queues; check whether XFF trust allows attacker-controlled key space; bound and evict, add a soak test; raise limits only after fixing growth.

Likely follow-ups: How would you reproduce in a test?

In this project: docs/troubleshooting/05.

Q110 [Scenario]. MySQL shows lock waits and the console times out. How do you investigate?

Check innodb_trx and data_lock_waits to find the blocking transaction, identify the code path (long transaction, missing index causing range locks, unsafe update pattern), kill it if necessary, then fix by shortening transactions, adding indexes, or changing lock order; optimistic locking reduces held locks.

Likely follow-ups: What isolation level matters? How do you avoid deadlocks?

In this project: Optimistic locking in ApiService; exercise 04 approach.

Q111 [Scenario]. Control-plane endpoints got slow after a migration. What now?

Compare query plans before/after (EXPLAIN ANALYZE), check new/dropped indexes, table statistics, lock contention from the migration, and ORM behaviour changes; roll forward with the right index or roll back the app (expand/contract keeps both working); add plan assertions for hot queries.

Likely follow-ups: How do you run big index builds safely?

In this project: V2 migration, QueryPlanIT.

Q112 [Scenario]. One gateway serves stale routes while others are fine. Why?

Check that gateway's snapshot age/version (readiness details, metrics), network policy or token errors to the control plane, repeated validation failures (INVALID_CONFIG) pinning an older snapshot, or a disk-cached snapshot loaded and polling failing. Fix connectivity/config; restart only if it has a valid cache or the control plane is reachable.

Likely follow-ups: How do you alert on version skew?

In this project: gateway_snapshot_age_seconds, SnapshotHealthIndicator.

Technical leadership

Q113 [Scenario]. How would you mentor a junior engineer who repeatedly introduces performance problems?

Diagnose together with evidence: profile their PRs, show the hot path, explain the principle behind it. Provide a short checklist for self-review, require before/after numbers for hot-path changes, pair on one optimisation, give a safe ticket with a measurable target, and review progress after a couple of weeks. Look for systemic causes (unclear SLOs, deadline pressure, missing tooling) and keep feedback specific, kind and actionable.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Likely follow-ups: What if it persists? How do you avoid becoming the bottleneck?

In this project: docs/leadership/mentoring-examples.md; performance-review checklist.

Q114 [Scenario]. How do you communicate release risk when a performance target is missed?

Lead with the decision needed, then facts (measured numbers vs target, scope affected), estimates (clearly labelled), mitigation (flag/canary/limits), rollback plan and owner, and the cost of delay. Offer options with consequences, state what is unknown, and confirm the decision in writing.

Likely follow-ups: How do you handle pressure to ship? What do you put in the exception record?

In this project: docs/leadership/stakeholder-and-release-communication.md.

Q115. How do you run an architecture review?

Circulate a short ADR draft in advance (context, options, trade-offs, risks, review triggers), time-box discussion around decisions and risks rather than preferences, record dissent, decide who decides, and capture follow-ups. Re-review when the stated triggers fire.

Likely follow-ups: How do you handle an HiPPO? What if consensus fails?

In this project: docs/adr/*, review-scenarios.md.

Q116 [Scenario]. Two senior engineers disagree about the design. What do you do?

Clarify the decision and criteria (requirements, risks, reversibility), ask each to state the other's position fairly, run a small experiment or spike if evidence is cheap, prefer reversible choices, document the decision and the dissent with review triggers, and move on with commitment.

Likely follow-ups: When do you override? How do you keep trust?

In this project: ADR template includes alternatives and review triggers.

Q117. How do you balance technical debt against feature delivery?

Make debt visible and priced (risk, cost of delay, interest), tie it to outcomes (incidents, lead time), reserve capacity (e.g. 15-20%), bundle debt with feature work in the same area, and track exceptions with owners and expiry dates.

Likely follow-ups: How do you justify to the business? What is not worth fixing?

In this project: Exception register in the vulnerability remediation doc.

Q118 [Scenario]. You are asked to certify production readiness of this platform. What do you say?

Provide an evidence-based assessment: what is ready (functional core, tests, live smoke), what is partial (resilience without chaos, observability stack not run), and gaps (Docker/Kubernetes/Helm/CI never executed, no live OIDC, Redis/MySQL tests skipped, no restore drill, no pen test). Recommend go-live blockers and a plan rather than a blanket yes.

Likely follow-ups: What are your top three blockers? What would you accept with an exception?

In this project: docs/leadership/production-readiness-assessment.md.

Q119. How do you facilitate a blameless post-incident review?

Focus on timeline, conditions and system safeguards rather than people; use "how did the system allow this" questions, identify detection and mitigation gaps, produce specific verifiable actions with owners and dates, and share lessons broadly. Track completion and revisit repeated causes.

Likely follow-ups: How do you handle a person-specific error? How do you measure improvement?

In this project: post-incident-review-template.md.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Q120. How do you onboard a new team to the platform?

Provide a self-service path: docs, a sample API, a checklist (owner, policy, target allowlist, consumer onboarding), a pilot with a champion, office hours, templates for policy requests, and feedback loops. Measure time-to-first-API and common support questions, then fix docs/tooling.

Likely follow-ups: What do you automate first? How do you handle exceptions?

In this project: Roadmap phase 3 self-service goals.

Q121. How do you estimate and plan a roadmap under uncertainty?

Slice vertically, estimate with ranges and confidence, plan the riskiest assumptions first (spikes), keep milestones outcome-based with exit criteria, track dependencies explicitly, and re-forecast regularly with actuals. Communicate assumptions rather than dates alone.

Likely follow-ups: How do you handle changing scope? What do you do about hard deadlines?

In this project: docs/leadership/delivery-roadmap-and-epics.md.

Q122. How do you keep code review standards consistent across a team?

Shared checklists and standards documents, PR templates, CODEOWNERS for sensitive areas, automated checks (lint, static analysis, tests) so humans review design and risk, calibration sessions with example PRs, and a clear definition of blocking vs nit. Review AI-assisted changes with the same bar and disclosure.

Likely follow-ups: How do you avoid review bottlenecks? What is a good review SLA?

In this project: docs/governance/code-review-checklist.md, PR template.

Appendix - quick reference

Topic	One-line answer to remember
Gateway design	Immutable route table + atomic config swap; control plane off the hot path; bounded everything.
Rate limits	Token bucket for APIs; atomic Lua in Redis; decide LOCAL vs CLOSED when Redis is down.
Policy propagation	Versioned snapshots, ETag polling, validate-then-swap, last-known-good; bounded, not instantaneous.
504 investigation	Split by code and target, compare timeout vs backend p99, check retries and breakers, revert recent changes first.
SSRF	Allowlist + blocked ranges + no redirects + re-check at connect + egress policy.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Topic	One-line answer to remember
Auth for many consumers	Per-consumer identity, hashed expiring keys or short-lived JWTs, subscription scoping, never forward credentials.
Cardinality	Labels only from bounded registries; detail goes to logs/traces.
JVM memory	Heap after GC, class histogram, heap dump; bound structures before raising limits.
Spec-driven AI	Spec -> design -> small tasks -> real verification -> independent review -> human approval.
Agent conflicts	Frozen contracts, file ownership, gates, evidence, escalation to architecture then human.
Context optimisation	Short always-on file + task modules + repo map + handoffs; measure tokens and pass rate.
Dependency patching	Evidence, reachability, smallest bump, regression tests first, canary, rollback; accept with owner/expiry if no fix.
Perf miss at release	Quantify, profile, fix or roll back, decide with owner; never ship on hope.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.